

Subprograme partea 2

Wednesday, May 6, 2026 6:56 PM

Definirea si declararea unor functii

Definirea = scrierea efectiva a functiei inainte de a o folosi

Declararea = „promiterea” (compilatorului) ca functia va fi implementata pana la finalul fisierului/structurii de fisiere

Apelarea = chemarea/lansarea in executie

Declararea variabilelor

Unde putem declara variabilele?

Variabilele se declara in memoria RAM (Random Acces Memory) care e impartita in 3 zone:

1. Zona de date (Data Zone) - unde se retin valorile constante => tipul de variabile constante cu key-word-ul `const`

`const int x = 5;` => nu putem modifica valoarea respectiva

2. **Zona de stiva (STACK)** - se retin toate variabilele, tablourile cu numar fix de elemente, functiile si apelurile de functie + executia

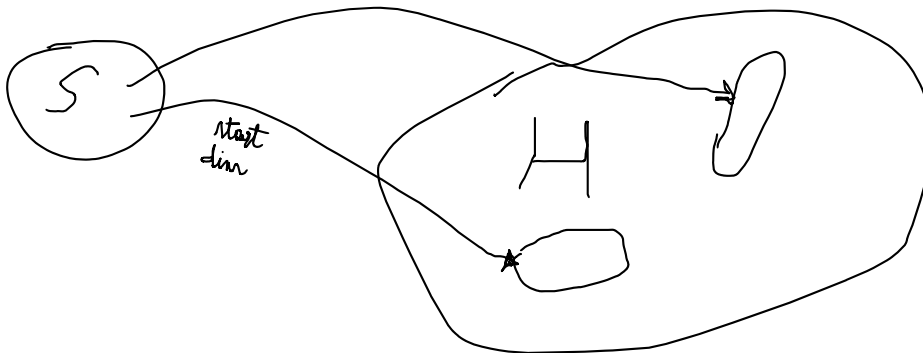
Avantaje: foarte rapida

Dezavantaje: dimensiune mica

3. Zona libera (HEAP) - se retin tablourile nealocate, fara un numar fix de elemente, se leaga de program prin stack

Avantaje: dimensiune mare

Dezavantaje: lenta



Prin ce se caracterizeaza:

1. Clasa de memorare = zona de memorie unde poate fi alocata
2. Vizibilitate = unde se poate folosi
 - a. La nivel de bloc -> intr-o anumita functie, intr-o anumita structura decizionala/repetitiva
 - b. La nivel de fisier -> vizibila in tot fisierul
 - c. La nivel de clasa -
3. Durata de viata = timpul in care ii este alocat spatiu de memorie variabilei
 - a. Statica = spatiul este alocat pe tot parcursul executiei programului
 - b. Locala = - | - pe parcursul executiei blocului in care a fost declarata
 - c. Dinamica = variabila se creaza -> I se alocata spatiu -> sa distrug/sterge variabila si sa eliberez memoria --- toate in timpul executiei
4. Tipul variabilei
 - a. Variabile locale - sunt initializate cu valori random
 - b. Variabilele globale - alocarea se face in zona de date + prioritatea: var locale > var globale cu acelasi nume si tip
 - c. Variabile statice - nu sunt sterse la finalul executiei functiei si prelucrarea functiei asupra lor va fi vizibila la fiecare apelare, initializarea se face la primul apel
`static int contor = 0;`

In functiile de tip void parametrii fac o copie superficiala (shallow copy??) locala a variabilelor locale din blocurile in care sunt apelate. Practic, modificarile aduse variabilelor de tip copie nu sunt vizibile si in variabilele date (transmitere prin valoare) => trebuie dat acces la zona de memorie a variabilelor unde vreau sa fac modificari (transmiterea prin referinta)

Transmiterea prin referinta

- Parametrul/parametrii functiei au acces la memorie
- Langa tipul de date la parametrii se pune &

Transmiterea parametrilor ca tablouri

- TABLOURI UNIDIMENSIONALE: In cadrul antetului pun parametrul `tip nume[]` - pot sa nu pun dimensiunea (nu e o problema daca o pun) si apelez cu `f(..., nume2, ...)`
- TABLOURI BIDIMENSIONALE: In cadrul antetului pun parametrul `tip nume[][dim_col]` - pot sa nu pun dimensiunea pt linii (nu e o problema daca o pun) si apelez cu `f(..., nume3, ...)`, dar trebuie pusa dimensiunea maxima posibila ($1 \leq nr_coloane \leq max_coloane \implies$ `tip nume[][max_coloane]`)

!!! Numele parametrilor nu trebuie sa fie la fel ca a variabilelor locale din anumita parte a codului si nu e o problema daca se intampla asta.

!!! Ordinea parametrilor ca tablouri in conteaza, adica nu trebuie sa ii pun la final in lista de parametrii

Functii cu parametrii impliciti

- Sunt functiile care imi permit o relaxare a dimensiunii listei cu valorile date la apelul functiei
- Asta nu inseamna ca pun doar parametrii impliciti
- Parametru implicit = parametru cu o valoare default data
- Daca in apelul functiei nu se cunoaste valoarea unei variabile sau nu este data in lista de apel, atunci se inlocuieste valoarea ei cu ce am implicit/initial
- Daca o functie are parametrii impliciti, atunci acestia vor fi la final

Tip_functie functie(tip_1 param_1, ..., tip_n param_n, tip_n+1 param_implicit1=val_1, ..., tip_m param_implicitm=val_m-n-1)

Apel functie functie(a1,a2,...,an, an+1)

